

软件过程改进方案设计文档

一、文档目的与设计依据

随着生成式 AI、大语言模型和智能体技术逐渐进入软件开发活动，软件过程改进不再只局限于阶段划分、角色分工和交付物规范，也开始关注如何利用 AI 提高需求整理、设计建模、编码实现、测试验证和问题分析等环节的效率。相关行业报告和开发者调查显示，生成式 AI 已经进入软件工程活动并影响开发者工具使用习惯，但其效果仍需要结合过程质量、交付稳定性和人工审核机制进行评估[5][6][7]。本设计文档以前一阶段完成的软件过程定义和《AI 技术应用场景匹配报告》为基础，进一步讨论 AI 技术如何融入既有软件过程，并形成软件过程改进方案。

在前一阶段的分析中，已经识别出五类较适合引入 AI 的应用场景，包括 AI 辅助需求规格说明书与用户故事生成、AI 辅助 UML 模型与系统设计草案生成、AI 辅助代码生成与代码审查、AI 生成测试用例与自动化测试脚本，以及 AI 日志分析与异常总结。因此，文档将会以需求分析、系统设计、编码实现和测试验证作为主要改进环节，同时将日志分析与异常总结作为过程改进阶段的扩展应用。

本方案的基本思路是：在保持原有软件过程阶段和人工评审机制的前提下，把 AI 作为辅助性工具嵌入具体活动节点。AI 主要承担资料整理、初稿生成、模型草案生成、局部代码辅助、测试用例扩展和结果摘要等工作；需求确认、架构取舍、代码合并、测试准入、缺陷关闭和版本发布等关键决策仍由项目成员负责。通过这种方式，AI 的输出可以成为提高过程效率的中间成果，但不会直接替代正式交付物。后续设计将重点围绕三个问题展开：第一，原有软件过程中的哪些活动适合引入 AI；第二，AI 介入后活动流程、角色职责和交付物会发生哪些变化；第三，如何通过人工审核、质量标准、版本记录和过程度量保证 AI 输出可控。本文档希望在效率提升和风险控制之间取得平衡，使 AI 技术真正服务于软件过程改进，而不是简单地停留在工具使用层面。

二、原有软件过程问题概述

原有阶段	主要痛点	AI改进目标
需求获取与分析	需求来源分散，人工整理耗时，需求表达不统一，用户故事和验收标准容易遗漏	使用AI生成SRS初稿、用户故事和验收标准，辅助识别模糊需求，并保留人工需求评审
系统架构与详细设计	UML图、接口说明和模块划分主要依赖人工起草，建模成本较高，设计方案比较不足	使用AI生成可编辑的UML草图、模块划分和接口草案，供设计人员评审修改
编码实现与单元验证	样板代码、接口适配和局部重构占用开发时间，人工代码审查容易遗漏低级问题	使用AI辅助局部代码生成和重构建议，并结合静态检查、单元测试和人工Review
集成测试与迭代修正	测试用例编写耗时，边界场景容易遗漏，自动化测试脚本准备成本较高	使用AI生成测试用例初稿、边界数据和自动化脚本草案，再由测试人员确认覆盖标准
项目总结与过程改进	缺陷记录和日志分析依赖人工汇总，复盘依据不足	作为扩展场景，使用轻量日志文件和LLM摘要辅助整理异常原因和改进建议

表1 原有软件过程问题与AI改进目标

三、AI增强软件过程总体设计

3.1 总体改进思路

改进后的软件过程将保留原有阶段划分，AI技术主要插入在以下四个关键位置：

- 在需求分析阶段，增加“AI需求辅助”节点，用于生成需求文档初稿、用户故事和验收标准，为后续需求评审提供基础材料。
- 在系统设计之后，增加“设计草案生成”节点，让 AI 根据已经确认的需求和设计约束生成 UML 草图、模块划分建议和接口草案，帮助设计人员更快形成可讨论的设计方案。
- 在编码实现过程中，允许开发人员使用AI进行局部代码生成、重构建议和初步代码检查，但代码是否被采纳仍取决于开发人员的理解、修改和测试结果。
- 在测试设计阶段，增加“AI测试生成”节点，用于根据需求说明、接口文档和代码片段生成测试用例、边界数据和自动化脚本草案，供测试人员进一步筛选和完善。

AI 生成的内容只是过程中的中间成果，必须经过相应阶段责任人的初审，再经过需求评审、设计评审、静态检查、单元测试、代码审查、测试覆盖检查等质量控制活动，才能进入正式版本或正式文档。

3.2 AI介入范围

AI 技术的介入范围主要限定在“辅助生成”和“辅助分析”两类工作。前者包括 SRS 初稿、用户故事、验收标准、UML 草图、接口草案、局部代码、重构建议、测试用例和测试脚本草案；后者包括模糊需求提示、代码审查建议、测试失败摘要和日志摘要等。

这样的范围划分是为了避免 AI 直接介入高风险决策。需求是否进入基线、架构方案是否采用、核心代码是否合并、测试结果是否通过、版本是否发布，仍然由项目成员按照原有过程进行确认。总结而言，AI 可以提供候选内容和辅助判断，但不能代替正式的软件工程决策。

3.3 人工保留职责

在 AI 技术介入后，原有角色的职责并没有被取消，而是增加了对 AI 输出进行判断和管理的任务。需求分析人员需要检查 AI 生成的需求是否有真实来源，是否符合业务目标，是否存在无依据扩展；设计人员需要判断 AI 生成的 UML 图、模块划分和接口草案是否符合系统约束；开发人员需要理解、修改并验证 AI 生成的代码，不能在未理解的情况下直接提交；测试人员需要评审 AI 生成的测试用例是否覆盖核心需求、边界条件和异常场景；项目负责人则需要关注各阶段的准入结果和过程度量数据。

3.4 软件过程重构前后对比

阶段	原有关键活动	改进后新增活动	新增交付物	新增质量控制点
需求分析	人工整理需求、编写SRS、维护需求跟踪矩阵	增加AI需求初稿生成、用户故事生成、验收标准补充	AI需求初稿、用户故事草案、AI需求审核记录	需求来源标注、人工确认状态、需求评审通过后才能进入需求基线
系统设计	人工划分模块、绘制UML图、编写接口说明	增加AI生成UML草图、模块划分建议和接口草案	Mermaid/PlantUML草图、AI设计建议记录	UML元素需追踪到需求，设计人员审核后才能进入系统设计说明书
编码实现	开发人员手写代码、调试、提交版本	增加AI局部代码生成、重构建议和代码解释记录	AI生成代码片段、重构建议、AI采纳与修改记录	静态检查、单元测试、人工Review均通过后才允许合并
测试验证	人工编写测试用例、执行测试、记录缺陷	增加AI测试用例初稿、边界数据和自动化脚本草案	AI测试用例草案、自动化脚本草案、覆盖检查记录	测试人员评审用例有效性，满足覆盖标准后才能作为正式测试用例
项目总结与过程改进	人工整理缺陷、日志和复盘材料	扩展使用LLM对日志和缺陷记录进行摘要	日志摘要、可能原因列表、复盘建议草案	根因结论必须人工复现确认，AI摘要不得直接作为正式结论

表2 软件过程重构前后对比

四、关键环节改进方案设计

4.1 需求分析阶段改进

需求分析阶段的主要任务是将用户访谈、功能设想、业务流程和约束条件整理为需求规格说明书、用户故事、用例说明和需求跟踪矩阵。原有流程主要依靠需求分析人员手工整理材料、归纳需求、编写 SRS，并维护需求追踪关系。这种方式能够保证人工判断，但在需求来源较多、表达不统一或边界条件不完整时，容易出现整理耗时、需求描述风格不一致、验收标准遗漏等问题。

引入 AI 后，需求分析人员仍然负责原始材料的筛选和需求确认，但可以先将访谈记录、功能清单、业务流程和已有需求模板输入 AI，由 AI 生成 SRS 初稿、用户故事、验收标准、模糊需求清单和需求追踪建议。生成式 AI 已被用于需求获取、需求分析和需求文档生成等需求工程活动，但相关研究也指出其输出仍需处理幻觉、可解释性和可复现性问题[8]。因此，AI 生成的内容不直接进入需求基线，而是作为需求评审前的辅助材料。需求分析人员需要逐条检查其来源、合理性和优先级，并对 AI 自行补充但缺少依据的内容标记为“待确认”或删除。

该阶段的改进重点在于将“从零编写需求文档”转变为“基于 AI 初稿进行审核、修订和确认”。这样可以缩短初稿形成时间，也有助于更早发现需求表述中的歧义和遗漏。正式准入标准应包括：每条需求必须有编号、来源、优先级和验收标准；核心需求能够追踪到后续设计和测试；AI 新增建议必须明确标注状态，不能直接作为已确认需求。

项目	内容
AI输入	访谈记录、功能清单、业务流程、约束条件、已有需求模板
AI输出	SRS初稿、用户故事、验收标准、模糊需求清单、需求追踪建议
人工审核重点	需求来源是否真实，是否符合业务目标，优先级是否合理，是否存在无依据扩展
质量标准	每条需求有编号、来源、优先级和验收标准；AI新增建议必须标注为待确认
度量方式	比较AI介入前后需求初稿编写耗时、需求歧义数量和人工修改比例

表3 需求分析阶段AI改进要素

AI改进需求分析流程

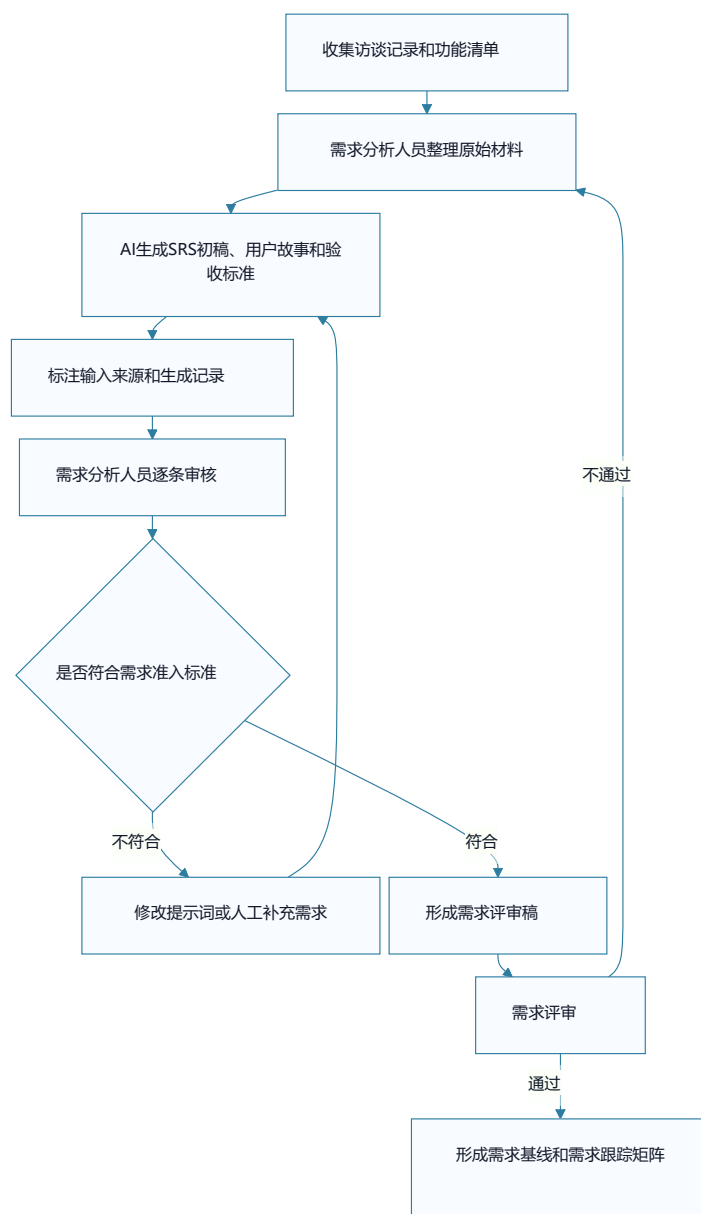


图1 需求分析阶段AI改进流程图

4.2 系统设计阶段改进

系统设计阶段需要在需求基线的基础上确定系统结构、模块划分、接口关系和关键数据结构。原有流程通常由设计人员阅读需求后手工划分模块、绘制 UML 图、编写接口说明并提交设计评审。该流程的优点是设计人员能够充分结合技术约束进行判断，但在文档起草和模型绘制方面工作量较大，尤其是流程图、类图、时序图和接口说明之间需要保持一致时，容易出现遗漏或表达不统一。

AI 介入后，可以由设计人员先明确系统边界、技术约束和核心设计目标，再使用 AI 生成 UML 草图、模块划分建议、接口草案和备选设计方案。尤其是在使用 Mermaid 或 PlantUML 这类文本化建模方式时，AI 生成的图形代码可以直接放入 Markdown 文档中，便于修改、评审和版本管理；Mermaid 官方文档也说明其适合用文本方式描述流程图等图形[1]。这样可以提高设计材料准备效率，也有助于团队在评审前比较不同设计思路。

不过，设计阶段的 AI 输出只能作为草案。模块边界是否清晰、架构是否可实现、接口是否完整、异常流程是否覆盖，仍必须由设计人员和项目负责人确认。正式设计文档中采用的 UML 图和接口说明，应能追踪到需求编号，并经过设计评审。对于核心业务规则、性能约束、安全约束或外部服务依赖，不能只依据 AI 建议确定，而需要结合原型验证或技术评审结果。

项目	内容
AI输入	需求规格说明书、用户故事、原型结论、非功能需求、技术约束
AI输出	模块划分建议、流程图、类图、时序图、接口草案
人工审核重点	架构是否可实现，模块边界是否清楚，接口是否完整，是否遗漏异常流程
质量标准	UML元素可追踪到需求编号；接口包含输入、输出和异常处理；设计评审通过后才准入
度量方式	比较设计文档准备时间、UML元素追踪率 and 设计评审发现问题数

表4 系统设计阶段AI改进要素

AI改进系统设计流程

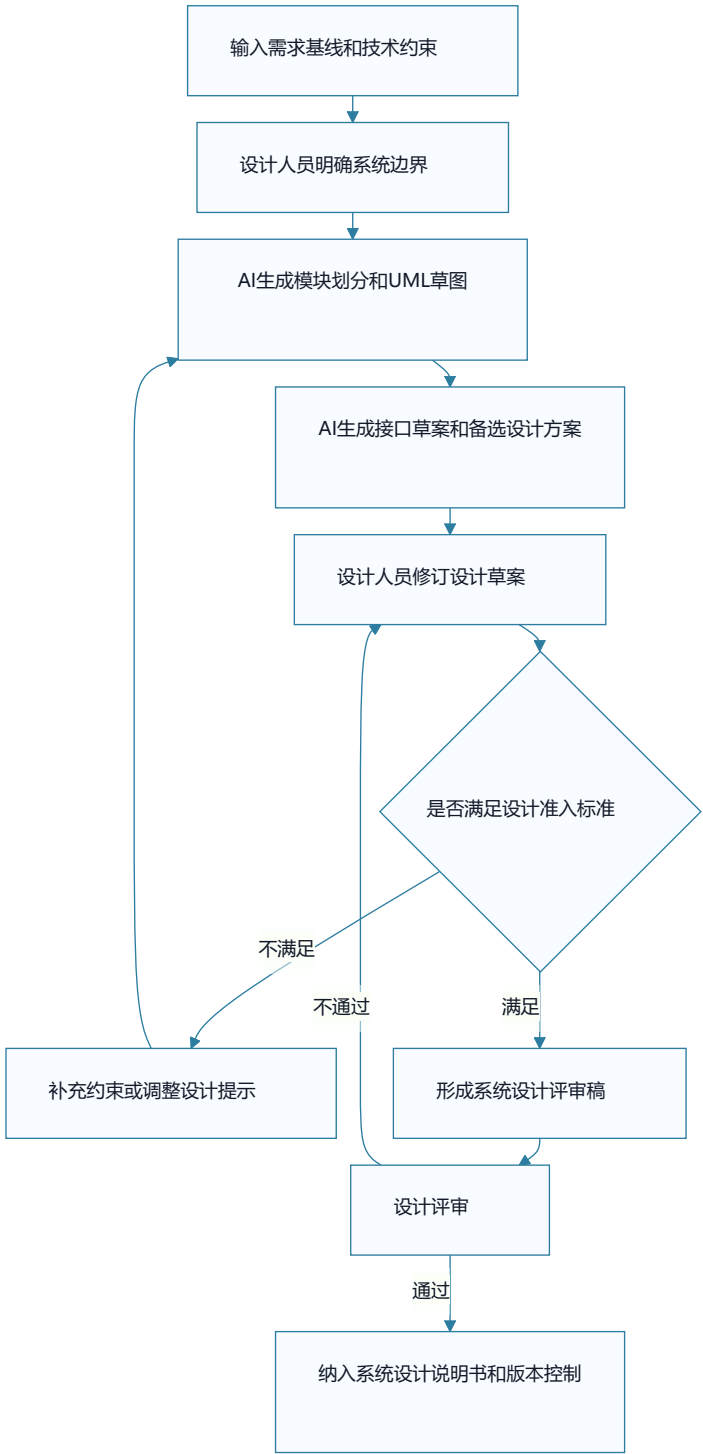


图2 系统设计阶段AI改进流程图

4.3 编码实现阶段改进

编码实现阶段的主要目标是根据设计文档完成模块代码、接口适配、数据处理和单元验证。原有流程主要依靠开发人员手写代码、调试、提交代码审查并执行单元测试。在课程项目中，很多代码并不是复杂算法，而是样板化的接口处理、参数校验、数据转换、异常处理和测试桩代码。这类工作重复性较强，容易占用开发时间，也容易在人工审查中遗

漏格式、异常处理或低级质量问题。

AI 在编码阶段适合承担局部辅助任务，而不是完成整模块开发。开发人员可以向 AI 提供需求编号、接口说明、代码上下文、编码规范和测试要求，让 AI 生成局部代码片段、重构建议、单元测试桩或代码解释。随后，开发人员必须阅读并修改 AI 输出，确认其符合项目结构和业务规则，再进行静态检查、单元测试和人工 Review。

该阶段的关键控制点是代码责任不能转移给 AI。无论代码片段是否由 AI 生成，最终提交人都必须能够解释代码逻辑，并对其质量负责。核心业务逻辑、安全相关逻辑、权限控制和数据一致性处理不应直接采用 AI 输出。只有通过静态检查、单元测试和人工 Review 的代码，才能合并到正式分支。

项目	内容
AI输入	需求编号、接口说明、代码上下文、编码规范、测试要求
AI输出	局部代码片段、重构建议、单元测试桩、代码解释
人工审核重点	开发人员是否理解代码，是否符合编码规范，是否存在安全风险，是否影响已有功能
质量标准	静态检查、单元测试和人工Review均通过；核心业务逻辑必须由开发负责人确认
度量方式	比较样板代码编写时间、AI代码采纳率、单元测试通过率和返工次数

表5 编码实现阶段AI改进要素

AI实现编码流程

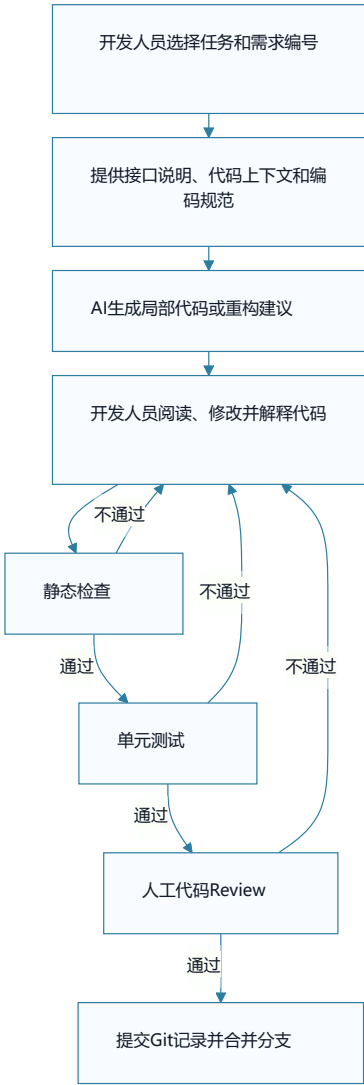


图3 编码实现阶段AI改进流程图

4.4 测试验证阶段改进

测试验证阶段是本方案中最适合作为后续原型验证的环节。原因在于测试活动的输入和输出都比较明确：输入可以是需求规格说明书、接口说明、设计文档或代码片段，输出可以是测试用例表、边界数据、异常场景和自动化测试脚本。相比需求和设计阶段，测试阶段的改进效果也更容易通过耗时、覆盖率、有效缺陷数和返工率等数据进行比较。

原有测试流程通常由测试人员阅读需求和接口说明后手工编写测试用例，再执行测试、记录缺陷并进行回归验证。这种方式依赖测试人员经验，容易遗漏边界条件和异常路径。AI 介入后，测试人员可以先确定测试目标和覆盖范围，再由 AI 生成测试用例初稿、边界数据、异常场景和 pytest、JUnit 或 Postman 脚本草案。之后，测试人员需要检查用例是否覆盖核心需求、是否存在重复无效用例、脚本是否可执行，并根据需求追踪矩阵补充必要场景。

该阶段的改进重点不是追求测试用例数量，而是提高测试设计的覆盖性和可验证性。正式测试用例必须经过人工评审，自动化脚本必须经过调试和执行验证，缺陷记录必须包含问题描述、复现步骤、责任人、修复状态和回归结果。后续原型验证可以选择一个模块，分别采用人工方式和 AI 辅助方式生成测试用例，对比用例生成耗时、需求覆盖率、有效缺陷数和人工修改比例。

项目	内容
AI输入	需求规格说明书、接口说明、设计文档、代码片段、历史缺陷
AI输出	测试用例表、边界数据、异常场景、pytest/JUnit/Postman脚本草案
人工审核重点	用例是否覆盖核心需求，是否重复无效，脚本是否可执行，缺陷记录是否完整
质量标准	每条核心需求至少有对应测试；异常和边界场景需覆盖；脚本执行结果纳入测试报告
度量方式	比较人工方式和AI辅助方式下的用例生成耗时、覆盖率和有效缺陷数

表6 测试验证阶段AI改进要素

AI生成测试用例与验证流程

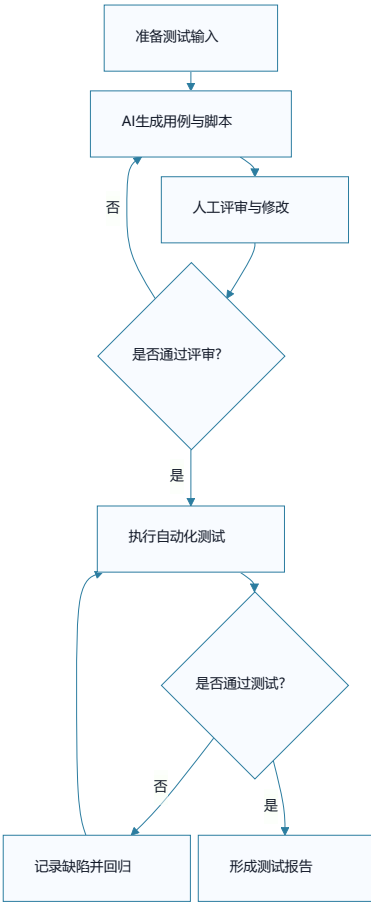


图4 AI生成测试用例与验证流程图

4.5 运维与过程改进阶段改进

测试、部署、演示和项目总结阶段会产生大量的日志、缺陷记录、错误信息和版本变更说明。原有过程通常依靠人工查看日志和缺陷记录，再根据经验撰写复盘总结。这种方式能够完成基本总结，但问题定位和改进建议容易受到个人经验影响，缺少较系统的数据依据。

本方案将日志分析与复盘总结作为扩展场景。团队可以在测试和演示过程中保留必要日志文件、错误信息、缺陷记录和版本变更说明，在脱敏后输入大语言模型，由其生成异常摘要、可能原因列表、排查建议和复盘草案。AI 的作用是帮助整理信息和形成初步分析，而不是直接判定最终根因。

该阶段的质量控制重点是证据和复现。AI 给出的结论只能作为“可能原因”，正式根因必须由开发人员或项目负责人根据日志、代码和复现实验确认。对于包含用户信息、密钥、接口地址或内部配置的日志，必须先脱敏再使用 AI 工具。最终进入项目总结的内容，应当是人工确认后的修复结论和过程改进建议。

项目	内容
AI输入	日志文件、错误信息、测试失败记录、缺陷记录、版本变更说明
AI输出	异常摘要、可能原因列表、排查建议、复盘草案
人工审核重点	是否有日志依据，根因是否可复现，建议是否可执行
质量标准	AI结论只能作为建议；正式根因必须人工验证；敏感日志需脱敏
度量方式	作为扩展观察项，记录故障定位时间和复盘材料整理质量

表7 运维与过程改进阶段AI改进要素

AI生成异常摘要流程

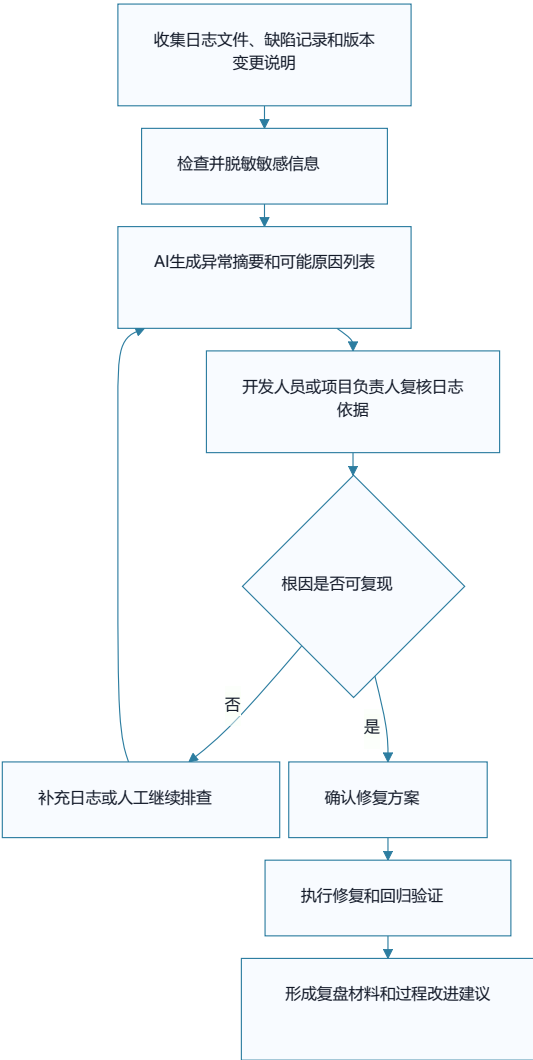


图5 运维与过程改进阶段AI改进流程图

五、AI工具链选型

本方案的工具链选型优先选择容易获取、能够嵌入实际开发过程、便于记录输入输出的工具。对于需求分析和系统设计阶段，通用大语言模型已经能够较好地完成文本整理、文档初稿生成和图形代码生成；而在编码实现和测试验证阶段，仅使用普通对话式工具已经不足以满足项目上下文理解、文件修改、测试运行和结果反馈的需要，因此更适合使用 Codex、Claude Code、Cursor Agent 等具备项目级上下文处理能力的编码 Agent。对于质量判断，则需要使用确定性的质量检查工具例如 SonarQube、CodeQL、Lint、pytest、JUnit 和 Postman，用于验证 AI 生成内容是否符合代码质量和测试执行要求。

阶段	推荐工具或技术框架	核心功能	与现有过程的集成方式	预期效率提升指标
需求分析	ChatGPT、Kimi；备选：通义千问、Claude	对访谈记录、功能清单和业务流程进行文本归纳，生成 SRS 初稿、用户故事、验收标准和模糊需求清单	在原有“需求收集—需求整理—需求评审”流程中，增加“AI 生成需求初稿”节点；需求分析人员逐条审核 AI 输出，并将确认内容写入需求规格说明书和需求追踪矩阵	预期缩短 SRS 初稿和用户故事编写时间，目标为减少约 30% 左右；同时需要观察需求歧义发现数量、人工修改比例和需求追踪覆盖率
系统设计	ChatGPT + Mermaid；备选：PlantUML、通义千问 + Mermaid	根据需求说明生成流程图、类图、时序图等文本化模型草案，并辅助生成模块划分和接口草案	在原有“设计建模—设计评审”流程前增加“AI 生成设计草案”节点；设计人员将 Mermaid 或 PlantUML 代码放入 Markdown 文档中修改，并在评审通过后纳入设计说明书	预期缩短 UML 草图和接口草案准备时间，目标为减少约 20%—30%；同时需要观察 UML 元素追踪率和设计评审发现问题数
编码实现	Codex、Claude Code、Cursor Agent；备选：GitHub Copilot、通义灵码	读取项目上下文，生成局部代码实现、修改函数、补充异常处理、提出重构建议，并生成单元测试桩	在原有“编码—调试—提交”流程中作为局部辅助工具使用；开发人员审查 AI 生成的 diff，运行测试后再决定是否提交	GitHub 关于 Copilot 代码质量的研究可作为 AI 编码辅助效果的参考[9]；本课程项目中主要观察 AI 代码采纳率、返工率和单元测试通过率
代码质量检查	人工 Review + SonarQube 或 CodeQL；备选：IDE 静态检查、Lint 工具	对代码进行静态扫描、质量规则检查、安全缺陷提示和代码异味识别；人工 Review 判断业务逻辑和设计一致性	在原有“代码提交—代码审查—合并”流程中增加静态检查节点；AI 生成代码和人工代码均需通过检查后才能进入合并流程；GitHub Copilot Code Review 可作为代码审查辅助工具[2]	代码质量不能只依赖 AI 判断，CodeQL 可用于代码扫描[3]，SonarQube 可用于代码质量规则检查[4]；指标包括静态检查问题数、Review 返工次数和合并前缺陷数

阶段	推荐工具或技术框架	核心功能	与现有过程的集成方式	预期效率提升指标
测试验证	Codex、Claude Code、Cursor Agent + pytest/JUnit/Postman；备选：ChatGPT、Kimi、通义千问，Testin XAgent、WHartTest 作为扩展工具	根据需求、接口和代码上下文生成测试用例、边界数据、异常场景和可执行测试脚本，并辅助分析测试失败结果	在原有“测试设计—测试执行—缺陷记录”流程中增加“AI 生成测试用例和脚本草案”节点；测试人员审核覆盖情况，运行 pytest、JUnit 或 Postman 脚本，并将结果写入测试报告	观预期缩短测试用例初稿和测试脚本草案生成时间，目标为减少约 30%—40%；同时需要观察需求覆盖率、边界场景数量、脚本可执行率和有效缺陷发现数
日志分析与复盘	日志文件 + LLM 摘要；备选：ELK、Grafana、AIOps 工具	对测试、部署或演示过程中的日志、错误信息和缺陷记录进行摘要，生成可能原因列表和复盘建议	在原有“缺陷记录—问题复盘—项目总结”流程中增加日志摘要辅助环节；日志需脱敏后输入模型，最终根因仍由开发人员复现确认	预期缩短日志摘要和复盘材料整理时间，目标为减少约 20%—30%；同时需要观察故障定位时间、根因建议采纳率和复盘建议有效性

表8 AI工具链选型

六、角色职责调整

AI 技术介入软件过程后，原有角色的职责需要相应调整。本方案在原有角色体系基础上增加 AI 工具负责人、AI 输出审核人和 AI 测试分析员三个职责角色。这些角色不一定作为独立岗位设置，而是可以由项目负责人、阶段负责人或测试人员兼任。

其中，AI 工具负责人主要负责 AI 工具清单、提示词模板、账号权限和数据脱敏规则；AI 输出审核人负责检查 AI 生成的需求、设计、代码和测试用例是否满足准入标准；AI 测试分析员则重点参与测试验证阶段，负责使用 AI 生成测试用例和脚本草案，并协助检查测试覆盖情况。与此同时，需求分析人员、设计人员、开发人员和测试人员仍然保留各自阶段的最终质量责任，只是在原有职责基础上增加对 AI 输出的检查、修改和记录工作。

角色	类型	主要职责	参与阶段	主要交付物
AI工具负责人	由项目负责人或配置管理员兼任	维护可用AI工具清单、提示词模板、账号权限、数据脱敏规则和AI使用记录	全阶段	工具使用说明、提示词模板、AI使用记录
AI输出审核人	由对应阶段负责人兼任	审核AI生成需求、设计、代码、测试用例是否满足准入标准	需求、设计、编码、测试	AI输出审核记录、采纳或退回意见
AI测试分析员	由测试人员兼任	使用AI生成测试用例初稿和脚本草案，检查覆盖率并整理测试结果	测试验证	测试用例草案、自动化脚本草案、覆盖检查记录
需求分析人员	原有角色扩展	检查AI生成需求来源、优先级、验收标准和追踪关系	需求分析	需求规格说明书、需求追踪矩阵
设计人员	原有角色扩展	修改AI生成UML图和设计草案，确认模块边界和接口约束	系统设计	系统设计说明书、UML模型、接口文档
开发人员	原有角色扩展	使用AI进行局部代码辅助，负责理解、修改、测试和提交	编码实现	源代码、单元测试报告、AI采纳记录
测试人员	原有角色扩展	评审AI生成测试用例，执行测试并记录缺陷	测试验证	测试用例表、缺陷记录、测试报告

表9 角色职责调整

七、AI输出物质量标准与准入机制

AI 介入软件过程后，生成的需求、设计、代码、测试用例和日志分析结果都不能直接作为正式交付物使用。为了保证过程可控，本方案对主要 AI 输出物设置准入标准。未通过审核的内容需要人工修改、补充材料或重新生成，不得进入正式文档、代码库或测试资产。

AI输出物	准入标准	验证方式	不通过时的处理
AI生成需求	有输入来源、需求编号、优先级和验收标准；AI自行扩展内容必须标记为待确认	需求评审、用户或项目负责人确认、需求追踪矩阵检查	退回修改，补充原始材料或删除无依据内容
AI生成UML图	模块、类、接口和流程能追踪到需求；图形语义清楚；可在Markdown中正常显示或可编辑	设计评审、需求到设计追踪检查、设计负责人确认	局部修改或重新生成，不进入正式设计文档
AI生成代码	开发人员能够解释代码；符合编码规范；通过静态检查、单元测试和人工Review	SonarQube、CodeQL或Lint检查；单元测试；人工代码审查	禁止合并，退回开发人员修改或重新生成
AI生成测试用例	覆盖核心需求、正常路径、异常路径和边界条件；重复无效用例可控	测试评审、需求覆盖检查、脚本执行验证	删除无效用例，补充提示词或人工补充用例
AI生成日志分析结论	有日志依据，结论使用“可能原因”表述，不直接当作最终根因	日志复核、问题复现、修复后回归测试	仅作为无效建议归档，不写入正式根因结论

表10 AI输出物质量标准与准入机制

八、风险评估与应对策略

风险类型	风险表现	影响环节	应对策略
AI幻觉	编造不存在的需求、接口、API、测试结论或根因	需求、设计、编码、测试、复盘	不允许将AI输出直接作为正式交付物；所有AI生成内容必须标注来源并经过阶段负责人审核
数据安全	上传密钥、隐私数据、未脱敏配置或敏感代码	全阶段	不允许上传密钥、隐私数据和未脱敏配置；输入AI前进行脱敏；只提供必要上下文
代码质量问题	AI生成代码存在漏洞、不可维护结构或与项目规范不一致	编码实现	AI代码必须经过静态检查、单元测试和人工Review；核心逻辑由开发负责人确认
测试用例重复或无效	测试用例数量多但覆盖不足，重复验证同一路径	测试验证	测试人员按需求追踪矩阵审核用例，删除重复用例，补充关键业务和异常场景
过度依赖AI	成员不理解AI生成内容，直接复制到文档或代码中	全阶段	要求记录人工修改意见；代码提交人必须能解释AI生成代码；设计和需求必须通过人工评审
工具访问限制	部分工具需要账号、付费、额度或特定网络环境，导致无法按计划使用	全阶段	每个AI场景至少准备一个可访问替代工具；保留人工流程作为兜底方案
版本管理混乱	多次AI生成导致文档、代码和测试用例版本不一致	文档、代码、测试	AI输出物纳入版本控制；记录生成时间、输入来源、采纳状态和修改人
验证数据不足	没有对比实验，无法证明AI确实带来过程改进	过程度量	原型验证阶段设计人工方式与AI辅助方式对比；记录耗时、覆盖率、缺陷数和返工率

表11 风险评估与应对策略

九、结论

本方案围绕前期软件过程定义和 AI 技术应用场景匹配结果，对原有软件过程进行了小规模的改进设计。方案将 AI 主要应用于需求初稿生成、UML 设计草案生成、局部代码辅助和测试用例生成等较容易落地的环节。日志分析与复盘总结作为扩展场景保留，用于支持项目后期的问题整理和过程改进。

从过程改进角度看，AI 的引入改变的是部分活动的执行方式，而不是原有软件过程的责任结构。需求、设计、编码和测试等阶段仍然保留人工评审和阶段准入机制，AI 生成内容只作为中间成果参与过程流转。需求分析人员需要确认需求来源和业务合理性，设计人员需要判断模型和接口方案是否可行，开发人员需要理解并验证 AI 生成代码，测试人员需要检查测试用例覆盖情况。通过这种安排，AI 能够提高文档起草、模型生成、局部编码和测试设计的效率，同时避免未经确认的内容直接进入正式交付物。

本方案还为 AI 输出设置了质量标准和风险控制要求。AI 生成的需求、设计、代码、测试用例和日志分析结果都需要保留输入来源、审核意见和采纳状态，必要时纳入版本管理。这样既能为后续复盘提供依据，也能降低 AI 幻觉、数据泄露、代码质量问题和版本混乱等风险。与单纯使用 AI 工具相比，将 AI 输出纳入软件过程管理，能够更好地保证过程的可控性和交付物质量。

参考文献

- [1] Mermaid. *Mermaid Documentation*. <https://mermaid.js.org/intro/>
- [2] GitHub Docs. *Using GitHub Copilot code review*. <https://docs.github.com/en/copilot/using-github-copilot/code-review/using-copilot-code-review>
- [3] GitHub Docs. *About code scanning with CodeQL*. <https://docs.github.com/en/code-security/code-scanning/introduction-to-code-scanning/about-code-scanning>
- [4] SonarSource. *SonarQube Server Documentation*. <https://docs.sonarsource.com/sonarqube-server/>
- [5] Capgemini Research Institute. *Turbocharging software with Gen AI: How organizations can realize the full potential of generative AI for software engineering*. 2024. <https://www.capgemini.com/insights/expert-perspectives/boosting-productivity-in-software-engineering-with-generative-ai>
- [6] Stack Overflow. *2025 Developer Survey: AI*. 2025. <https://survey.stackoverflow.co/2025/ai>
- [7] Google Cloud DORA. *Accelerate State of DevOps Report 2024*. 2024. <https://dora.dev/research/2024/dora-report/>
- [8] Cheng H., Husen J. H., Lu Y., Racharak T., Yoshioka N., Ubayashi N., Washizaki H. *Generative AI for Requirements Engineering: A Systematic Literature Review*. arXiv, 2024/2025. <https://arxiv.org/abs/2409.06741>
- [9] GitHub Blog. *Does GitHub Copilot improve code quality? Here's what the data says*. 2024. <https://github.blog/news-insights/research/does-github-copilot-improve-code-quality-heres-what-the-data-says/>